

OWASP Top 10

2025

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Broken Access Control

Índice

Introducción

¿Qué significa "Broken Access Control"?

¿Por qué ocurre esta vulnerabilidad?

Ejemplos reales y típicos

¿Qué puede hacer un atacante si explota esto?

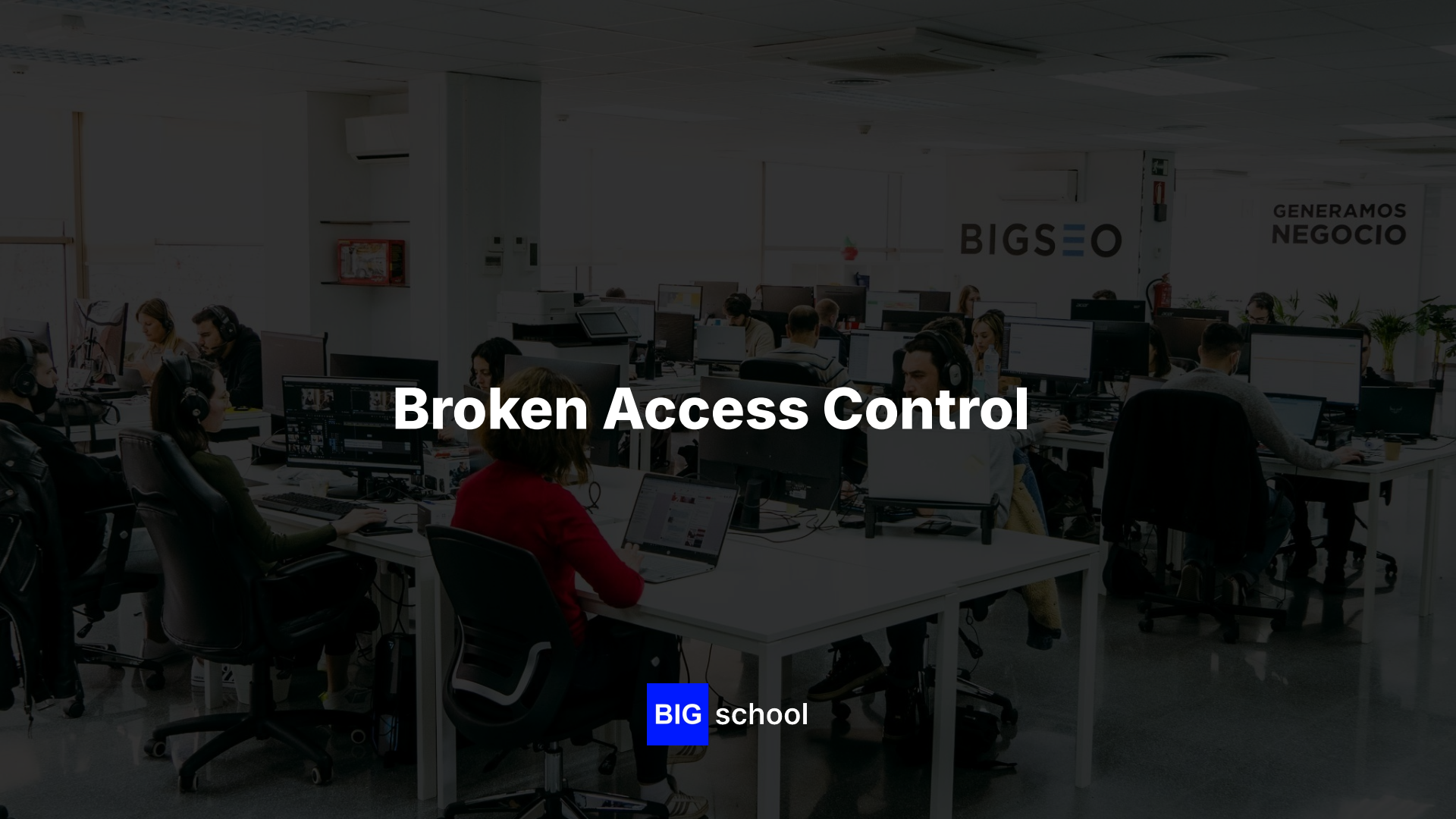
¿Cómo detectarlo?

¿Cómo prevenirlo?

¿Cómo mitigar si ya estás afectado?

Checklist rápido

Conclusiones



Broken Access Control

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Introducción

- Vulnerabilidad #1 en OWASP Top 10:2025
- Fallo crítico al validar quién puede hacer qué sobre qué recurso
- Afecta directamente confidencialidad, integridad y disponibilidad
- Objetivo: entender riesgos, detectar, prevenir y mitigar desde el diseño

¿Qué significa "Broken Access Control"?

se produce una falla al comprobar permisos antes de ejecutar una acción

- Tres preguntas clave: ¿Quién? ¿Qué permiso? ¿Sobre qué recurso?
- Validación inconsistente o ausente en el servidor
- Regla clave: autorización siempre en servidor, nunca en cliente

¿Por qué ocurre esta vulnerabilidad?

- Validación solo en frontend o cliente
- Confiar en parámetros de URL (user_id, role=true) sin verificar
- Reglas de autorización dispersas o inconsistentes entre endpoints
- Roles demasiado amplios o herencia de privilegios no justificada
- Sesiones/tokens mal gestionados o reutilizados

Ejemplos reales y típicos

- IDOR: acceso directo a objetos (/invoices/1234) sin verificar permisos
- Rutas admin expuestas sin comprobación de rol
- Escalada de privilegios por roles mal definidos o configurados
- Confusión: autenticado $\neq$ autorizado (login ok, pero acción sin permiso)
- Parámetros de cliente dictando acceso (isAdmin=true desde frontend)

¿Qué puede hacer un atacante si explota esto?

- Exposición o modificación no autorizada de datos privados
- Escalada de privilegios (usuario → administrador)
- Ejecución de acciones administrativas o borrado de registros
- Pivoteo lateral usando datos o sesiones comprometidas
- Impacto directo en cumplimiento normativo y reputación

¿Cómo detectarlo?

- Desarrollo: code review, tests de acceso cruzado, SAST/DAST, threat modeling
- Producción: logs de acceso (user-id + recurso-id), alertas de patrones anómalos
- Señales: 200 OK inusuales a recursos sensibles, errores 500 en checks de permisos
- Auditoría: pentesting enfocado en autorización y matrices de acceso por endpoint

¿Cómo prevenirlo?

- Principios: mínimo privilegio, deny-by-default, autorización centralizada
- Validar siempre en servidor; nunca usar parámetros de cliente para decidir permisos
- Modelar roles claros (RBAC/ABAC) y separar endpoints administrativos
- Tokens/sesiones seguros: verificar pertenencia y evitar reutilización
- Middleware/guards de autorización reutilizables en todas las rutas

¿Cómo mitigar si ya estás afectado?

- Contener: restringir endpoint afectado o aplicar reglas WAF temporales
- Investigar: revisar logs, identificar usuarios y datos comprometidos
- Rotar credenciales/sesiones potencialmente expuestas
- Parchear: implementar validación server-side + tests de bloqueo
- Auditar y monitorear: pentest focalizado, pruebas regresivas y alertas

Checklist rápido

- ✓ ¿Autenticación y autorización validadas en servidor en cada endpoint?
- ✓ ¿Aplicado principio de mínimo privilegio y deny-by-default?
- ✓ ¿Permisos independientes de parámetros enviados por el cliente?
- ✓ ¿Logs registran quién accedió a qué recurso y cuándo?
- ✓ ¿Tests automáticos en CI/CD validan accesos no autorizados?

Conclusiones

- Vulnerabilidad #1, pero altamente evitable con disciplina de diseño
- Autorización centralizada y validación server-side son innegociables
- Como desarrollador: middleware, tests de acceso y logs trazables marcan la diferencia
- En 2025: tratar la autorización como regla inquebrantable en cada commit
- Pequeñas validaciones evitan incidentes de alto impacto



¡MUCHAS GRACIAS!

BIG school

BIGSEIO

**GENERAMOS
NEGOCIO**